

PANDSY

GTM Engineer

The GTM system, built and explained

14,575 leads

2,887 websites built

218 cities, 48 states

$637.99 spent

4 months, 2 people

The lead pipeline

Workflow A, the engine that turns an empty sheet into a scored, contactable, brand-matched lead. 54 nodes, self-hosted on n8n. Everything that follows describes **one run**, because the run is the unit that repeats all day.

What a perfect lead means

A US auto repair shop, electrician, or plumber with no working website, where the pipeline can recover a phone number, an email address, a logo, plus the font used in that logo.

Everything the pipeline does serves that one definition. A lead missing the logo is still usable. A lead missing the phone is dropped.

What one run does

A run picks up where the last one stopped. It takes the next 100 cities from the list, searches all three trades in each of them, then filters what comes back down to businesses with no real website.

From there it recovers contact details from Facebook, pulls the brand out of the logo, scores each lead, writes every row into the sheet, then records what the run itself cost.

RUN PARAMETERS

Run cadence	One run every 50 minutes
Cities per run	100, taken in population order
Trades per run	3 (auto repair, electrical, plumbing)
Place searches per run	~300, at $0.032 each
Concurrency	One run at a time, enforced
Nodes in the workflow	54
Search credits per lead	~2.75 on average
Written per run	One row per lead, one row in the log

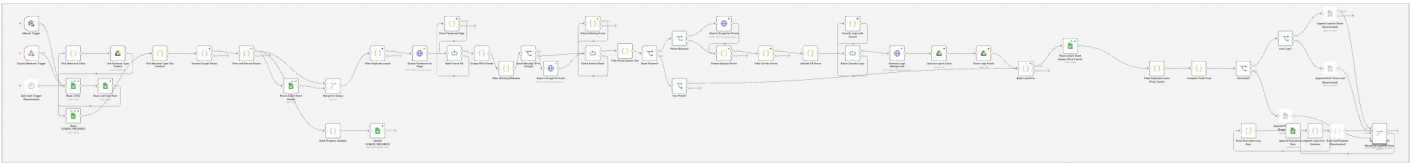
UNIT COSTS INSIDE A RUN

Google Places: Business search	$0.032 per call
Scrappa: Search credit	$0.000303 per credit
Gemini: Logo classification	~$0.000568 per call
ScrapingBee: Page fetch	$0.0002 per credit
Photoroom: Background removal	$0.02 per image
Cloudinary: Logo cropping	Free tier

THE SEVEN STAGES OF A RUN

1. Starting a run
2. Choosing where to look
3. Sourcing businesses
4. Removing duplicates
5. Recovering contacts
6. Extracting the brand
7. Writing it down

WORKFLOW A, FULL CANVAS, 54 NODES



1 Starting a run

Trigger, lock, run number

Run every 50 minutes
Scheduled trigger

One run starts every 50 minutes
Two other triggers sit alongside it: a manual button for testing, plus a webhook I can fire to force a run on demand.

One run at a time
Concurrency lock

Never let two runs overlap
Without this, two executions could interleave writes into the same sheet. The pipeline is single-threaded on purpose.

Claim a run number
Execution log, direct API call

Take the next run number
Reads the last run number from the execution log with exponential backoff, 2 seconds up to 60. It bypasses the built-in Sheets node, which hits a 60 call per minute quota under load.

2 Choosing where to look

Cities, trades, coverage

Load the city list
2,203 cities, population order

Load the city universe
2,203 US cities held in population order, each carrying its own state.

Take the next 100 cities
Rolling city walk

Move to fresh ground
A stored index advances 100 cities each run then wraps at the end, so runs move across the country instead of re-scraping the same places.

Load the three trades
Google Drive folders

Pick the business types
Plumbing, auto repair, electrical. Each folder also holds the design assets the site builder draws from later.

Check what is covered
Coverage ledger

Skip ground already walked
One record per city, state, trade combination, so a run does not repeat work a previous run already finished.

Build the search list
Trade, city, state pairs

Assemble this run's queries
Produces the exact combinations the run sends to the search API.

3 Sourcing the businesses

Search, then filter hard

Search Google Places
~300 searches, $0.032 each

Find every candidate business
The largest single cost inside a run, which is exactly why every filter that follows runs before any paid enrichment.

Check for a listed website
First website test

Drop anyone Google already has online
Google Places returns a website field whenever a business has one on file. Any result with that field filled is dropped on the spot, before it costs a cent to enrich. This catches the obvious cases cheaply.

Test the site that is found
Second website test, stage 5

Judge whether the site actually works
A missing website field does not prove a business has no site, so the deeper test comes later. Stage 5 fetches whatever address the Facebook page lists, then reads the response: dead, tiny, or parked pages count as no website, a live page with real business signals removes the lead.

Apply the quality filters
Fit gates

Drop anything that is not the customer
Rating between 2.5 and 5, at least four ratings on file, a rating of 4.0 or better unless four reviews in the sample already sit at four stars or higher, a state matching the city requested, a business name present.

4 Not paying twice for the same lead

Deduplication before spend

Read the existing database
Live sheet read

Pull every business already stored
Name, address, city, state for every row already written, merged against this run's fresh results.

Remove duplicates early
First deduplication pass

Drop repeats before spending a credit
Matching on business name, address, city, state together. Running deduplication here rather than at write time is what keeps the search bill proportional to new leads rather than to raw volume.

Log coverage, then stop
Parallel side branch

Record what this run covered
A branch that writes back which combinations were covered, then ends. It never rejoins the lead flow.

5 Recovering contact details

Facebook first, search second

Find the Facebook page
One search credit, always

Locate the business on Facebook
Facebook is the richest single source for these trades. One credit per lead, spent every time, because it feeds the email, the phone, the logo at once.

Process in batches
Failure isolation

Keep one bad page from killing the run
Splitting the run into batches means a single unparseable page cannot take the whole execution down.

Confirm the right business
Distinctive-token match

Reject lookalike pages
Candidate pages are scored so generic words like plumbing or electric never produce a match on their own. This rule came from wrong-business matches I had to clean out of the sheet by hand.

Pull the email from Facebook
Before any paid search

Take the email off the page itself
Always attempted first when a Facebook URL exists, since it costs nothing extra.

Test their listed website
5 second fetch, then judge

Prove the business has no usable site
Non-2xx responses, tiny pages, parked domains all count as no website. Real business signals on a live page remove the lead entirely. This is the gate the whole product rests on.

Pay only when needed
Conditional gate

Skip the paid search if an email exists
A small condition with a large effect once the run count grows, because it removes a credit from most leads.

Search for a missing email
Paid fallback, validated

Fall back to search
A domain allowlist, a distinctive-token match, a platform blocklist decide whether a found address really belongs to this business. Two extra fallback queries were removed once they stopped paying for themselves.

Search for a missing phone
Rarely needed

Fill a gap Places left
Places usually returns a phone already, so this branch is the exception rather than the rule.

Drop unreachable leads
Phone validation

Require a working phone number
Anything outside 10 or 11 digits is dropped. A lead with no phone has no fallback channel, so it is not worth carrying forward.

6 Turning a business into a brand

Logo, colors, font

Skip logos with no email
Cost gate

Do not pay to brand an unreachable lead
Skips the vision model entirely when no email was recovered, since that lead can never be emailed anyway. At target volume this one condition holds the model bill near $2.70 a day.

Verify it is a real logo
Vision model, two stages

Separate logos from photos
Facebook blocks datacenter addresses, so the page HTML comes through a proxy. Stage one rejects three locked categories: a photograph of a sign, a marketing flyer, a composed scene. Stage two returns dominant colors, a font guess from a 40 font whitelist, a bounding box, a quality score.

Crop to the logo mark
Image transform

Trim away the empty frame
Fires only when the logo fills less than 85 percent of the image. Any failure returns the original rather than breaking the run.

Remove the background
Transparent PNG

Make it usable on any color
Costs $0.02 an image, returning a cut-out the generated website can place anywhere.

Host the logo
Shared by link

Give the site builder a URL
Stored per business, readable by link, so the site generator can reference it directly.

7 Writing it down

Score, store, log, sort

Price and score the lead
Cost from credits used

Work out what this lead cost
Computes the real cost of this lead from credits actually consumed, then sets its score from which of phone, email, logo, font were recovered.

Check duplicates again
Second deduplication pass

Catch anything added mid-run
By business name, address, city, state together, then separately by email, since the sheet may change while the run is still going.

Total the run cost
Aggregation

Roll it up
Combines per-lead costs into a single figure for the run.

Route by what was found
Three write branches

Send each lead down its own path
Leads with an email plus a logo, email-only leads, phone-only leads each take a different branch.

Write the rows
Append to the database

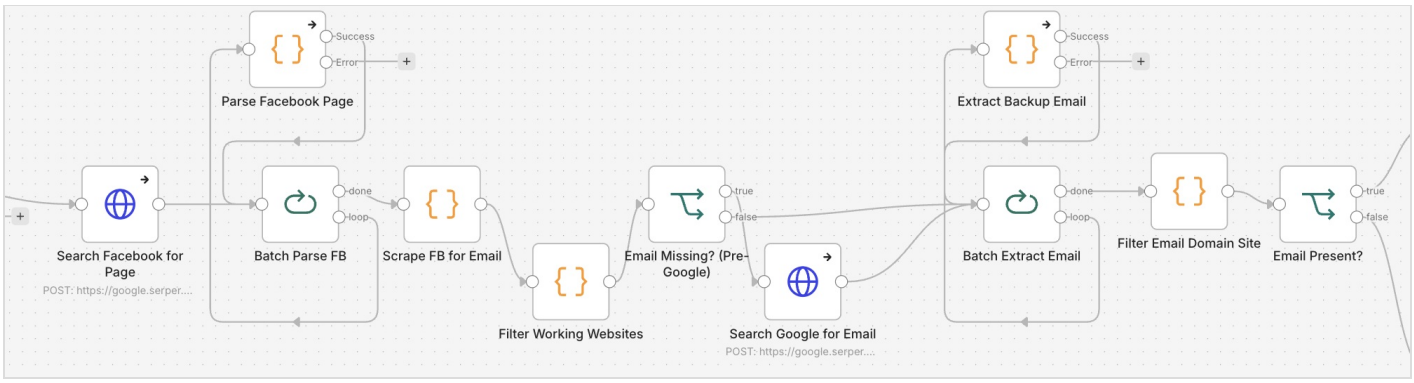
Place every lead in the sheet
The branches merge back into one stream, then the rows are inserted into the right part of the sheet.

Log the run
One row per execution

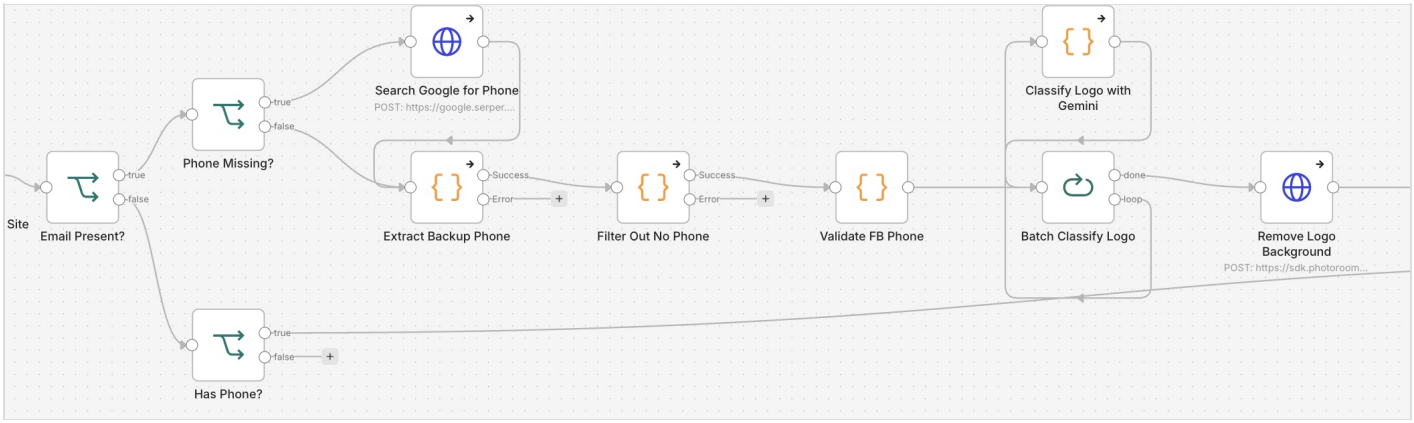
Record what the run itself did
Credits consumed, total cost, count of leads per score tier with cost broken out per tier. This record is what makes per-lead economics visible rather than theoretical.

Re-sort the database
Score, then run number

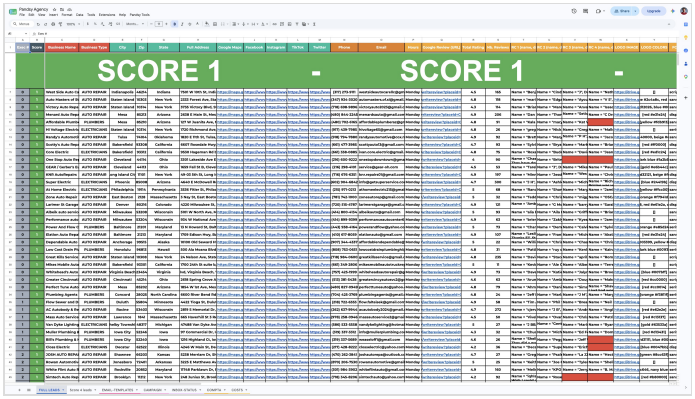
Keep the best leads on top
Pinned example rows are held at the top of the sheet, with everything else ordered by quality.



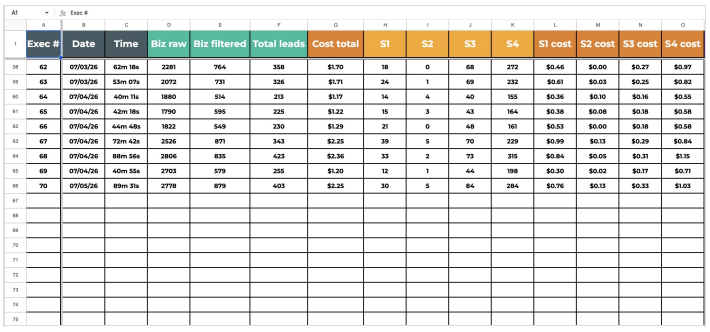
Stage 5 in the live canvas: finding the Facebook page, scraping the email, testing the listed website, then falling back to search only when the email is still missing.



Stage 5 into stage 6: the phone fallback with its validation, then the batched logo classifier feeding background removal.



The lead database. One row per business, banded by score.



Exec #	Date	Time	Biz raw	Biz filtered	Total leads	Cost total	S1	S2	S3	S4	S1 cost	S2 cost	S3 cost	S4 cost
62	07/03/26	62m 18s	2281	764	358	$1.70	18	0	68	272	$0.46	$0.00	$0.27	$0.97
63	07/03/26	53m 07s	2072	731	326	$1.71	24	1	69	232	$0.61	$0.03	$0.25	$0.82
64	07/04/26	40m 11s	1880	514	213	$1.17	14	4	40	155	$0.36	$0.10	$0.16	$0.55
65	07/04/26	42m 18s	1790	595	225	$1.22	15	3	43	164	$0.38	$0.08	$0.18	$0.58
66	07/04/26	44m 48s	1822	549	230	$1.29	21	0	48	161	$0.53	$0.00	$0.18	$0.58
67	07/04/26	72m 42s	2526	871	343	$2.25	39	5	70	229	$0.99	$0.13	$0.29	$0.84
68	07/04/26	88m 56s	2806	835	423	$2.36	33	2	73	315	$0.84	$0.05	$0.31	$1.15
69	07/04/26	40m 55s	2703	579	255	$1.20	12	1	44	198	$0.30	$0.02	$0.17	$0.71
70	07/05/26	89m 31s	2778	879	403	$2.25	30	5	84	284	$0.76	$0.13	$0.33	$1.03

The execution log. One row per run, with cost split by score tier.

Four tiers, set by what the pipeline recovers

Score 1	Phone, email, logo, font recovered. The perfect lead.	718
Score 2	Phone, email, logo. No font identified.	79
Score 3	Phone, email. No logo recovered.	3,195
Score 4	Phone only. Not emailable.	10,583

Counts on the right are cumulative across every run. Only the first three tiers are emailable. Score 4 leads carry no email address, so they were useless to an email product, but we kept them anyway in case we wanted to reach those businesses another way later, such as calling them.

THE RUNS, ADDED UP

What the pipeline produced in total

Leads	Leads in 24 hours	Websites built	Email opens
14,575	**720**	**2,887**	**40.7%**

What I would do differently

1. **I built before I asked.** No customer interviews came before the build. Volume was the part I knew how to engineer, so I engineered it first. Twenty conversations with shop owners would have cost a week, and would have tested the offer before 2,887 sites existed.
2. **Draw it, plan it, then build it.** I executed fast without holding the whole picture in my head. The cost arrived later: changing one part made other parts fall over, because nothing had been designed to fit together. Mapping the full system on paper first would have prevented most of that.
3. **Building it all with AI made it fast to write, expensive to trust.** The failures were silent rather than loud: duplicate leads slipping past deduplication, a cost constant left pointing at the old provider's rate so every logged figure came out wrong. My speed of building outran my ability to verify what I had built.
4. **Being cheap with tools cost more than it saved.** It was our own money, so we took the cheapest option at every layer. That choice showed up directly in the quality of what came out. Paying more for better tools would have bought better results than the savings were ever worth.
5. **I would protect my sleep.** I was obsessed with the pipeline, running on three to four hours a night, no exercise, barely eating. In that state I broke things while fixing others, then changed parts that never needed changing. My judgement, not the tooling, was the real bottleneck. I loved the work, which is exactly why I would guard my health next time rather than spend it.
6. **The unknowns are known now.** Most of what slowed the build was not in the plan, it was the problems that only appear once you are building: blocked requests, quota limits, images arriving in the wrong shape. None of those would surprise me a second time, so rebuilding this system would go far more smoothly than building it did.